ALACRITY / FLUX · مقالهٔ پژوهشی · ۱۰ مهر ۱۴۰۵

از حافظهٔ یک‌میلیون‌توکنی تا پاسخِ قابل بررسی

روایت طراحی، آزمون و بازنگری حافظه، مدیریت توکن و تصمیم‌گیری در Flux؛ از v1 تا v5

گزارش تحقیق و توسعه · ۲ اکتبر ۲۰۲۶ · همراه با نمودارها و خروجی‌های واقعی

چکیده. این تحقیق بررسی می‌کند که انتخاب دقیق شواهد و کنترل پاسخ تا چه اندازه می‌تواند کیفیت یک مدل کوچک‌تر را بهبود دهد. مسیر کار از فشرده‌سازی یک مخزن بزرگ به گراف آغاز شد و به معماری سه‌جزئی v5 رسید. در یک پایلوت واقعی با ۲۰ سؤال مشترک، Luna + v5 به ۱۶ پاسخ درست و Luna + v4 به ۱۰ پاسخ درست رسید؛ زمان متوسط نیز حدود ۳۰٪ کاهش یافت. بهبود عمدتاً در سؤال‌های وابسته به حافظه دیده شد و فاصلهٔ استدلال مستقل با مدل قوی‌تر هنوز بسته نشده است.

در این مقاله

مسئله: آیا حافظهٔ بهتر می‌تواند فاصلهٔ مدل‌ها را کم کند؟

از گراف فشرده تا بازیابی یک‌گذره

معماری v5: سه مسئولیت، یک قرارداد مشترک

چگونه آزمایش کردیم؟

نتیجه‌ها: بهبود حافظه، محدودیت استدلال

پاسخ‌های واقعی؛ پشت هر عدد چه اتفاقی افتاد؟

این کار چه چیزی برای Flux به ارمغان می‌آورد؟

منابع، قابلیت بازتولید و حدود ادعا

مسئله: آیا حافظهٔ بهتر می‌تواند فاصلهٔ مدل‌ها را کم کند؟

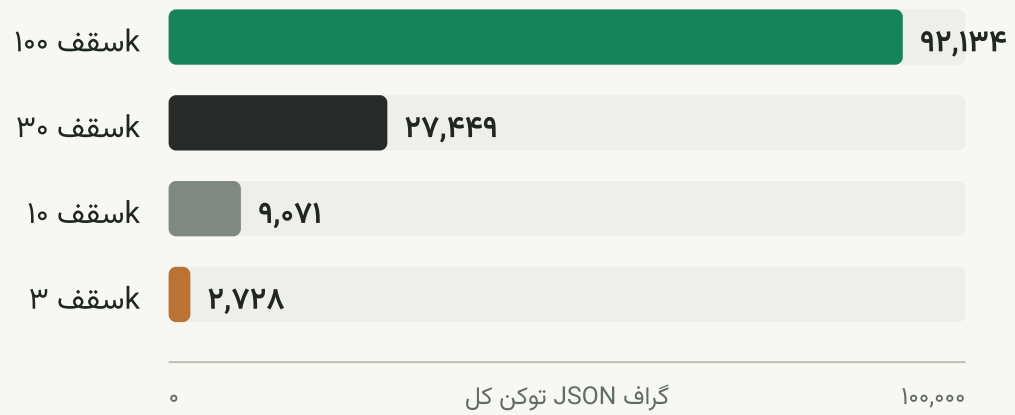
این تحقیق از یک فرض مشخص شروع شد: شاید بتوان با ساختن حافظه‌ای بزرگ، فشرده‌کردن آن به یک گراف و انتخاب اطلاعات مناسب برای هر درخواست، کیفیت پاسخ یک مدل کوچک‌تر را بالا برد. هدف اولیه ساختن مخزن یک‌میلیون‌توکنی و رساندن نمایش گرافی آن به حدود صد هزار توکن بود. مسئلهٔ اصلی اما صرفاً اندازهٔ حافظه نبود؛ باید می‌فهمیدیم چه اطلاعاتی در فشرده‌سازی از بین می‌روند و کدام شواهد برای پاسخ‌دادن ضروری‌اند.

یک میلیون توکن؛ گرافی زیر صد هزار توکن

ایدهٔ شما این بود که منابع را بزرگ و کامل جمع کنیم، سپس با ارزش‌گذاری و ترکیب ارزش، آن‌ها را بلوک‌بندی و فشرده کنیم؛ بلوک‌های مشترک بین یال‌ها نیز نقش یادآوری داشته باشند. «صد هزار» حجم کل گراف بود، نه تعداد گره.

۱	۲	۳
منبع ۱M	**گراف ≥۱۰۰k**	**بستهٔ درخواست**
۱۰۰ بلوک خامِ ۱۰هزارتوکنی	نمای مشتق‌شده، همراه reference	فقط شواهد مرتبط برای مدل

v1: اندازهٔ واقعی نمایش گراف در چهار سطح



سطح گراف	شواهد باقی‌مانده	سؤال کامل فقط از گراف
۹۲,۱۳۴ توکن	۳۰۰۰ از ۱۱,۳۱۲	۱۳۶ / ۳۴
۲۷,۴۴۹ توکن	۸۷۰ از ۱۱,۳۱۲	۱۳۶ / ۹
۹,۰۷۱ توکن	۲۹۰ از ۱۱,۳۱۲	۱۳۶ / ۴
۲,۷۲۸ توکن	۸۷ از ۱۱,۳۱۲	۱۳۶ / ۰

نتیجه: کوچک شدن گراف، حفظ همهٔ اطلاعات نبود. منبع اصلی باید بماند؛ جبران اطلاعات حذف‌شده از آرشیو بیرونی انجام می‌شود، نه از «فهم» یا آموزش پنهان مدل.

مقیاس 1M روی دادهٔ مصنوعی بود. snapshot واقعی ۱۱۷ :Flux فایل و ۲۵۰,۷۵۲ توکن. diffusion در v1 یک random walk با ۲۴ گام بود، نه مدل diffusion مولد. [S1]

از گراف فشرده تا بازیابی یک‌گذره

نسخه‌ها را نباید پنج پلهٔ قطعیِ بهبود دانست. هر نسخه یک فرض را آزمود و گاهی نشان داد که پیچیده‌ترکردن الگوریتم به نتیجهٔ بهتر نمی‌رسد. در نسخه‌های نخست، وزن‌دهی و انتشار روی گراف بررسی شد؛ سپس تمرکز به انتخاب شواهد، هزینهٔ هر درخواست و در نهایت اعتبارسنجی دقیق پاسخ منتقل شد. نمودارهای این بخش نتیجه‌های همان آزمایش‌ها را نشان می‌دهند و چون مجموعه‌های آزمون یکسان نیستند، یک رتبه‌بندی مستقیم میان تمام نسخه‌ها ارائه نمی‌کنند.

پنج نسخه؛ پنج تصمیم قابل‌ردیابی

v1
graph + diffusion

یادگیری: آرشیو را حذف نکنیم

۴۷۶۰ اجرای retrieval؛ صفر تماس مدل در مرحلهٔ اولیه

v2
value + shared blocks

یادگیری: وزن دلخواه تضمین بهبود نیست

۷۸۸۴ اجرای retrieval؛ صفر تماس مدل

v3
request RAG + E5

یادگیری: pruning حجم را کم کرد، استدلال را نه

۱۴۴ تماس واقعی در آزمایش تاریخی

v4
snapshot + one pass

یادگیری: سرعت خوب، verifier ناقص

۱۵۶ سؤال offline؛ اجرای مدل در pilot بعدی

v5
typed proof + ledger

یادگیری: نوع شاهد و repair اهمیت دارند

۵۰ تست؛ ۲۴۰ offline؛ سپس ۲۲ تماس Luna

نسخه	حافظه / گراف	توکن / تصمیم
v1	خلاصهٔ extractive، شباهت واژگانی، archive fallback	rank و بازیابی گراف
v2	ماتریس ارزش و reference مشترک	ارزش / هزینه؛ شاهد رابطه
v3	RAG مخصوص درخواست، embedding واقعی E5، note و bridge	fixed-1M یا adaptive؛ planner/critic دسته‌ای؛ prune
v4	index مشترک، scope، lexical با regex، یک hop	یک گذر، router دوحالته، verifier زیررشته‌ای
v5	AST/schema، هویت، scope و proof typed	router سه‌حالته، bundle لازم، ledger، admission

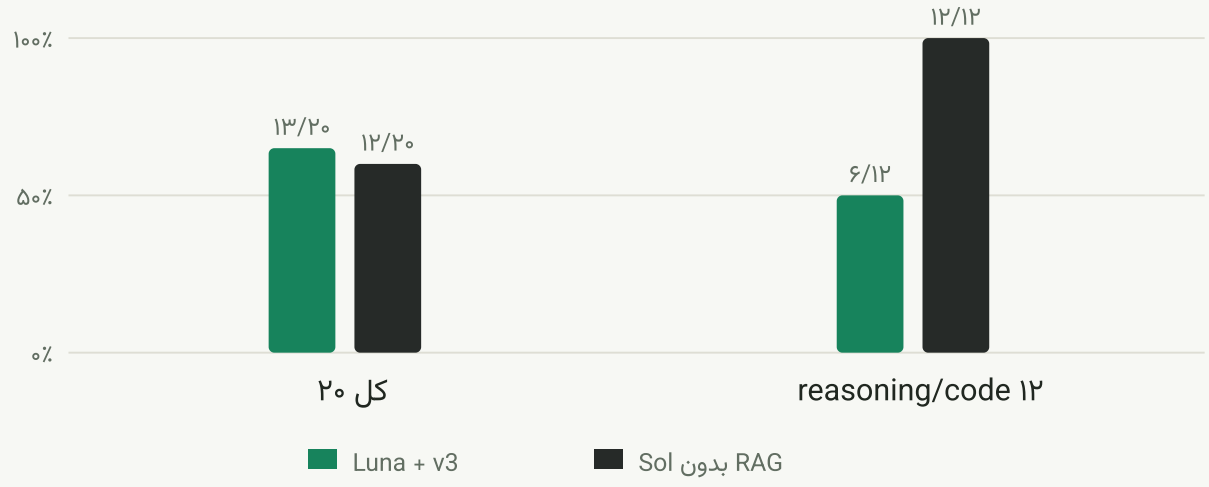
> سؤال اصلی از «چقدر بیشتر compact کنیم؟» به «کدام شاهد دقیقاً برای این نیاز لازم است و چه زمانی حق تأیید داریم؟» تغییر کرد.

در v2، شواهد کامل واقعیِ وزن‌دار ۱۵/۲۰ بود و گراف قبلی ۱۶/۲۰؛ خود وزن‌دهی برتری نداد. در v1 نیز diffusion نسبت به کنترل بدون diffusion مزیت مشخص نداشت. [S1, S2]

v3: RAG کمک کرد؛ فاصلهٔ reasoning بسته نشد

در v3، E5 واقعی با بردار ۳۸۴بعدی و پنجره‌های پوشش‌دهندهٔ ورودی بلند استفاده شد. planner و critic روی دستهٔ سؤال‌ها اجرا شدند. منبع، نوت، bridge و pruning به بستهٔ درخواست تبدیل شدند.

v3 تاریخی: دادهٔ کامل در برابر زیرمجموعهٔ اطلاعات یکسان



مسیر تاریخی	کل ۲۰ سؤال	دانش اختصاصی ۸	reasoning/code ۱۲
Luna بدون RAG	۷	۰	۷
Sol بدون RAG	۱۲	۰	۱۲
Luna + v3 fixed	۱۳	۷	۶
Sol + v3 fixed	۱۹	۷	۱۲

کاهش بستهٔ ۸ سؤال دانشی

کاهش input مرحلهٔ پاسخ

۶۱٫۵۴٪

۲۲۶۵٫۳۸ → ۸۷۱٫۲۵ توکن

۲٫۷۳٪

scaffold و تماس‌های دیگر مهم‌اند

> Luna ۱۳/۲۰ در برابر Sol ۱۲/۲۰ بدون RAG، به معنی Luna قوی‌تر نیست: دسترسی به دانش اختصاصی برابر نبود. در مسائل با اطلاعات یکسان، Sol هر ۱۲ را حل کرد و Luna+v3 شش مورد را.

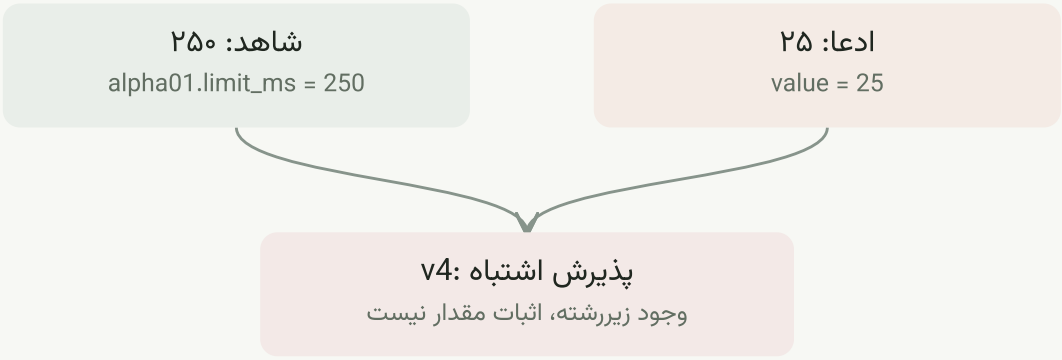
۱۴۰ پاسخ در ۷ arm، دو تماس planner/critic و دو probe؛ جمع ۱۴۴ تماس. ۵ تا ۷ تماس برای هر درخواست، شرح دقیق این اجرای v3 نبود. نتیجه‌های این صفحه به آزمایش تاریخی مربوط‌اند. [S3]

v4: حذف کار تکراری، همراه با چند خطای مهم

ایندکس، هزینهٔ واحدها و رابطه‌ها یک بار برای snapshot ساخته شدند. درخواست به یک گذر lexical، انتخاب معیار و یک hop محدود شد. در گزارش ۱۵۶تایی، زمان آماده‌سازی میانه از حدود ۱۹۱۰ ms به ۱٫۱۸ ms رسید؛ این مقایسهٔ offline بود، نه زمان کل تولید جواب.

سنجهٔ گزارش قدیمی ۱۵۶تایی	v3 fixed در harness v4	v4
شواهد کامل	۱۵۵ / ۱۵۶	۱۵۶ / ۱۵۶
میانهٔ آماده‌سازی	۱۹۱۰٫۲۷ ms	۱٫۱۸ ms
میانگین توکن بسته	۲۵۳۳٫۸۹	۳۲۴٫۲۸
کیفیت خروجی مدل	اندازه‌گیری نشد	اندازه‌گیری نشد

اشکال verifier زیررشته‌ای



ایراد بازتولیدشده	چرا مهم است؟
موجودیت یا field اشتباه	وجود مقدار در شاهد متعلق به نیاز نبود
پاسخ خالی / criterion گم‌شده	نبود خطا را موفقیت کامل حساب می‌کرد
citation غایب	fallback به کل بسته، اتصال ادعا را از بین می‌برد
math صحیح بدون RAG	پاسخ صحیح به‌دلیل نبود در متن رد می‌شد
budget=1 / حافظهٔ خالی	بستهٔ ۴۴توکنی یا خطای empty vocabulary

سرعت retrieval به‌تنهایی کیفیت پاسخ را ثابت نمی‌کند. baseline گزارش v4 با نسخهٔ نهایی مدل‌آزمایی‌شدهٔ v3 یکسان نبود؛ encoder و مراحل planner/critic تفاوت داشتند.

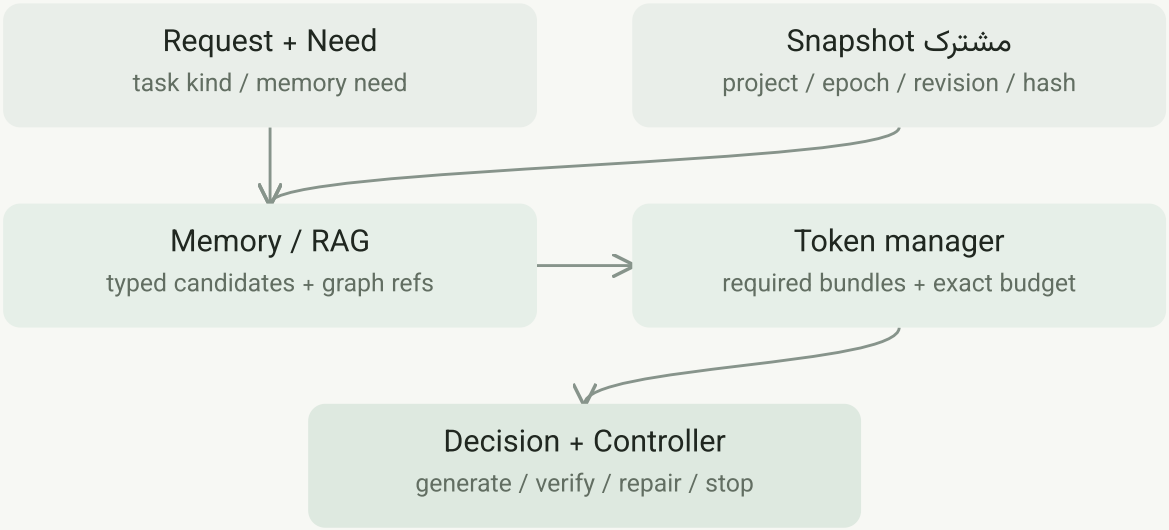
۹ regression شناخته‌شده بعداً در v5 بررسی شد. این‌ها holdout تازه یا درصد بهبود کیفیت عمومی نیستند. [S4, S5, S7]

معماری v5: سه مسئولیت، یک قرارداد مشترک

نتیجهٔ بررسی معماری این بود که حافظه، مدیریت توکن و تصمیم‌گیری باید مسئولیت‌های مستقل داشته باشند، اما از یک قرارداد مشترک استفاده کنند. اگر هر سه در یک الگوریتم بزرگ ادغام شوند، تشخیص علت خطا دشوار می‌شود. اگر کاملاً مستقل و بدون قرارداد مشترک باشند، ممکن است انتخاب حافظه با بودجهٔ توکن یا معیار پذیرش پاسخ ناسازگار شود. در v5، Controller این هماهنگی را انجام می‌دهد: ابتدا نوع درخواست و شواهد لازم مشخص می‌شوند، سپس بسته زیر بودجه ساخته می‌شود و در پایان پاسخ با معیار متناسب با همان درخواست بررسی می‌شود.

سه الگوریتم جدا؛ یک قرارداد و یک Controller

مسیر اصلی v5



ماژول	مسئولیت	خروجی
Memory	lexical، AST/schema، canonicalization، scope و relation	واحدهای eligible و gaps
Token	حفظ bundle لازم، dedup و trim گروهی optional	Pack یا budget_infeasible
Decision	route و انتخاب نوع repair، checker یا توقف	receipt صریح
Controller	ترتیب اجرا، ledger و idempotency	حداکثر دو تلاش فیزیکی

رتبهٔ بالا، scope و proof را دور نمی‌زند. اگر یک نیاز ضروری مبهم، فاقد شاهد یا بیش‌ازبودجه باشد، مدل قوی‌تر جای منبع معتبر را نمی‌گیرد.

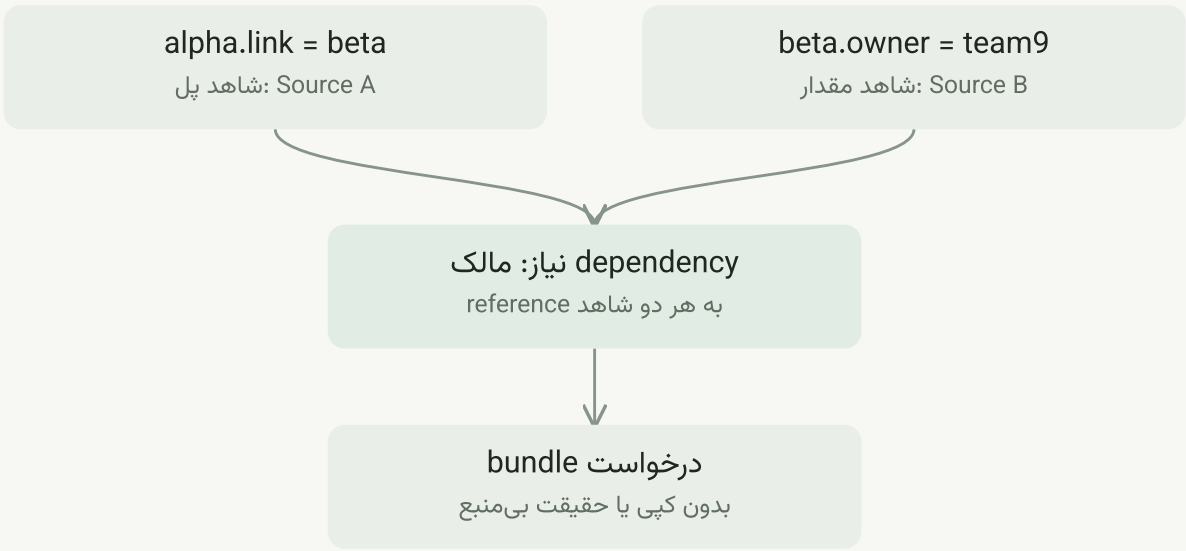
درخواست، نوت نیاز است؛ حقیقت تأییدشده نیست. خروجی مدل نیز برای حافظه فقط proposal با reference و hash تولید می‌کند. پذیرش انسانی جدا از وضعیت verifier باقی می‌ماند.

کد اجرایی: verifier_v5.py، engine_v5.py، memory_v5.py، contracts.py و controller_v5.py؛ هرکدام در آزمایش مستقل. [S6, S7]

مینی‌بلوک مشترک: یک منبع، چند reference

Snapshot قبل از index با revision، epoch، project و status پالایش می‌شود. متن Python معتبر با AST، JSON با schema و رکورد متنی با grammar ثبت‌شده استخراج می‌شود. اطلاعات نامفهوم به‌صورت متن می‌ماند؛ با حدس به fact تبدیل نمی‌شود.

مثال شماتیکِ یک hop ثبت‌شده



ماتریس ارزش v5؛ پس از عبور از gateها

مولفه	قاعده / وزن	آیا آموخته‌شده است؟
هویت	entity + namespace صحیح؛ وزن ۸	خیر
field	فیلد نیاز با شاهد یکسان؛ وزن ۴	خیر
اعتبار	role و proof معتبر؛ وزن ۲	خیر
هزینه	جریمهٔ ثابت ۰٫۰۰۱ × cost	خیر
context اختیاری	امتیاز lexical برای قطعات محدود	خیر

> این ماتریس در trace ثبت می‌شود؛ وزن‌ها ثابت‌اند. فشرده‌سازی و reference مشترک، آموزش پارامتری یا یادگیری وزن از بازخورد نیست.

یال‌ها: explicit entity reference، adjacency، containment و bare-name call حل‌شده در همان فایل. default یک hop از via_field است؛ زنجیرهٔ دلخواهِ چند field و note معنایی بین همهٔ یال‌ها پیاده نشده‌اند. [S6, S7]

شاهد لازم را برای جا شدن قربانی نمی‌کنیم

۱

واحد کامل

هزینهٔ هر واحد پیشاپیش

۲

bundle لازم

مقدار + پل + بدنهٔ موردنیاز

۳

بسته exact

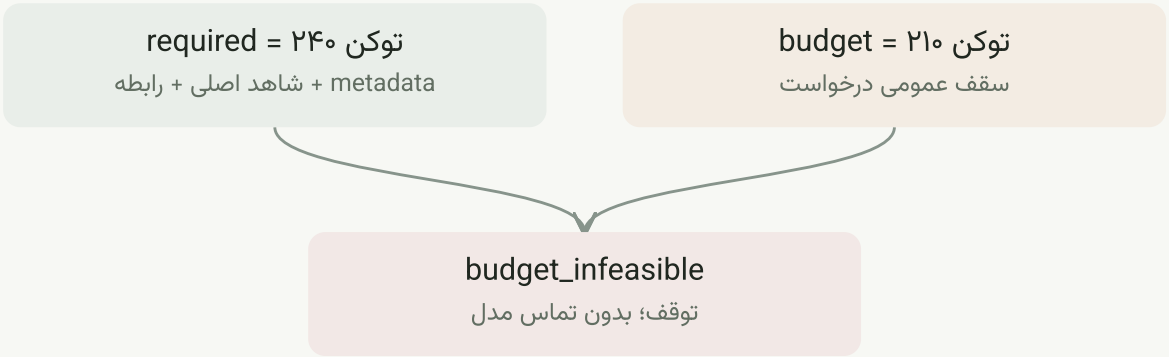
حداکثر دو شمارش serialization

۴

prompt admission

قرارداد + سؤال + pack

مثال شماتیک بودجه: required بزرگ‌تر از سقف



کار مجاز

- dedup referenceها و واحدها
- یک حذف گروهی از context اختیاری
- حفظ دو سر یال و تمام requiredها
- رد prompt بزرگ پیش از provider

کار غیرمجاز

- بریدن بی‌صدای شاهد لازم
- حذف پل و نگه‌داشتن مقدار بی‌منبع
- پر کردن آرشیو تا ۱M برای هر سؤال
- یکی دانستن pack با input مدل

لایهٔ شمارش	تعریف
آرشیو / snapshot	فضای نگه‌داری؛ کل آن به مدل فرستاده نمی‌شود
pack	JSON شواهد، واحدها، binding و relation
prompt محلی	دستور + Request + pack؛ admission مستقل
provider input	مصرف واقعی ثبت‌شده؛ شامل scaffold CLI و تماس repair

در pilot واقعی، input کل v5 برابر ۳۷۹,۴۵۲ بود؛ این عدد از ۲۲ تماس می‌آید. میانگین pack نهایی ۱۹۶٫۳ توکن است و با context کل provider برابر نیست.

توکن‌های محلی: o200k_base. هزینهٔ دلاری از receipt اشتراک معلوم نبود؛ «صفر» یا صرفه‌جویی دلاری ادعا نشده است. [S7, S8]

تصمیم از نوع مسئله و نوع proof می‌آید

project_grounded

وجود نیاز منبع یا اشارهٔ مثبت به پروژه → بازیابی

self_contained

قرارداد بستهٔ ریاضی یا کد → بدون شواهد خارجی

uncertain

تشخیص ناکافی → probe کوچک؛ بدون فرضِ خودبسنده بودن

پاسخ	روش بررسی	مرز verified
fact typed	value / نوع / / entity / field / namespace citation	همان ادعا و همان snapshot
literal-set	تطابق مجموعهٔ کامل؛ membership فقط اگر صریح باشد	همان constraint
ریاضی ساده	حساب محدود روی expression عمومی	فقط عملگرها و ورودی‌های پشتیبانی‌شده
کد	اجرای تست عمومی در worker محدود	فقط caseهای اجراشده
reasoning پیچیده / متن آزاد	بدون checker معتبر: unverified	درستی معنایی ادعا نمی‌شود

۱

تولید اول

receipt، Luna کامل

۲

بررسی

شکست معتبر یا وضعیت صریح

۳

یک repair

diagnostics عمومی، بدون gold

۴

توقف نهایی

هیچ تولید سوم پنهانی نداریم

مثال واقعی: repair نامناسب v4

`["enabled_2","disabled_2"]` پاسخ اولیهٔ صحیح ↓ verifier زیررشته‌ای رد کرد ↓ `null` پاسخ نهایی غلط، اما v4 پذیرفت

در اجرای نهایی، v4 دو پاسخ factual صحیح را پس از repair از دست داد. در v5 دو repair، دو سؤال natural-alias را به پاسخ درست رساندند.

Luna-only policy در pilot؛ Sol و escalation استفاده نشد. حالت provider ناقص/اثر نامعلوم خودکار retry نمی‌شود. worker کد sandbox سخت‌شدهٔ OS یا سقف قطعی RAM نیست. [S7, S8]

چگونه آزمایش کردیم؟

برای سنجیدن الگوریتم، دو سطح آزمون لازم بود. آزمون offline نشان می‌دهد چه شواهدی بازیابی شده‌اند، چه مقدار توکن مصرف شده و مسیر آماده‌سازی چقدر طول کشیده است. این آزمون به‌تنهایی دربارهٔ درستی پاسخ مدل چیزی ثابت نمی‌کند. به همین دلیل، پس از انجماد نسخهٔ نهایی، یک مقایسهٔ واقعی میان Luna + v5 و Luna + v4 اجرا شد. در این مقایسه، سؤال‌ها مشترک بودند و پاسخ نهایی با مرجع یا آزمون اجرایی مستقل ارزیابی شد.

۲۴۰ درخواست؛ شواهد بیشتر با metadata بزرگ‌تر

۱۱۲۶۰ واحد، شش خانوادهٔ ۴۰تایی، دو warmup و پنج تکرار جفت‌شده با ترتیب تصادفی. canonical unitهای یکسان به هر دو موتور داده شد. نتیجهٔ صحیح checker از fixture آماده سنجیده شد؛ این ۲۴۰ خروجیِ تولیدی LLM نبود.





شواهد کامل

۱۵۰ / ۱۵۰

v4: ۱۴۰ / ۱۵۰

route مطابق قرارداد

۲۴۰ / ۲۴۰

v4: ۱۹۸ / ۲۴۰

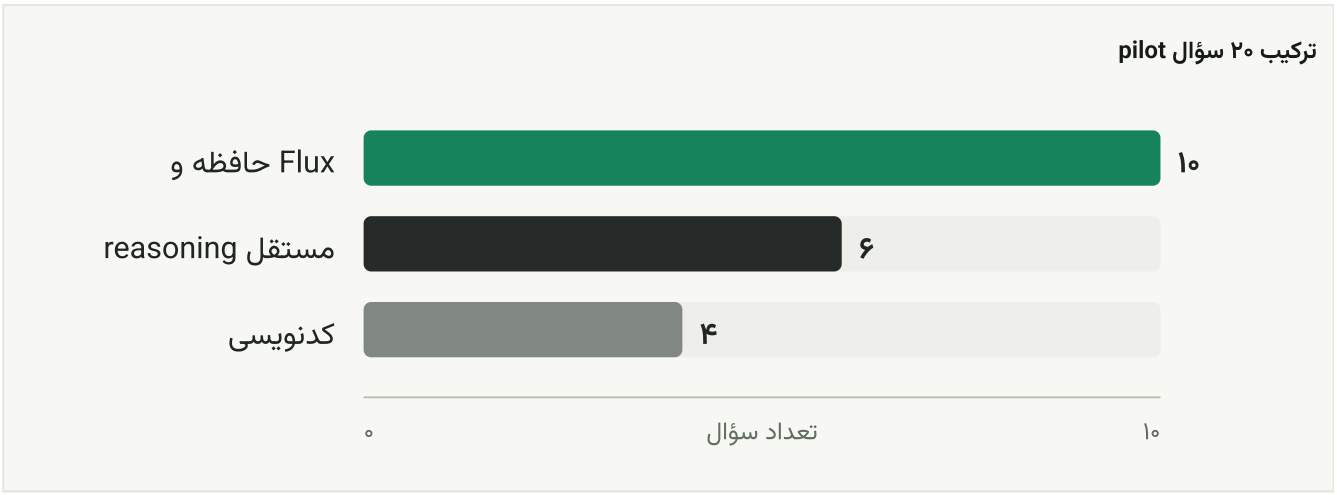
خانواده	v4 شواهد کامل	v5 شواهد کامل
حقیقت دقیق	۴۰ / ۴۰	۴۰ / ۴۰
alias عمومی	۴۰ / ۴۰	۴۰ / ۴۰
رابطه	۱۰ / ۲۰	۲۰ / ۲۰
freshness / scope	۳۰ / ۳۰	۳۰ / ۳۰
کد وابسته به منبع	۲۰ / ۲۰	۲۰ / ۲۰

بده‌بستان واقعی ٪۳۳٫۱۵ offline: زمان میانهٔ کمتر، اما ٪۱۱۸٫۷۱ بستهٔ بزرگ‌تر. metadata نوع و استناد رایگان نیست. زمان ساخت v5 حدود ۵۵۹ ms و v4 از واحد آماده حدود ۴۷۰ ms بود.

۲۰ missing، ۱۰ ambiguous و ۶۰ مسئلهٔ مستقل جدا از ۱۵۰ شاهد مثبت بودند. ۵۰ تست پاس و ۸۰ case عمومی کد اجرا شد. r1 پس از خطای parser واقعی کنار گذاشته و source آن حفظ شد؛ اجرای نهایی با seed 981315 منجمد شد. [S7]

از «شاهد کامل» به «جواب واقعی درست»

بعد از درخواست شما، ۲۰ سؤال مشترک برای هر دو نسخه آماده شد. مدل یکسان `gpt-6-luna` با `reasoning=low` و حساب موجود Codex اجرا شد. schema و verifier مختص نسخه بودند؛ ارزیابی مستقل، gold را بعد از تولید استفاده کرد.



۱	۲	۳	۴
Freeze	**Generate**	**Receipt**	**Evaluate**
کد، jobs، source و gold	دو worker؛ حداکثر یک repair	turn.completed، exit=0، بدون ابزار	gold جدا و تست‌های withheld

کنترل آزمایش	چیزی که واقعاً انجام شد
منابع مشترک	canonical unitهای v5 برای هر دو engine
حافظه	۸ سؤال مصنوعی + ۲ سؤال قرارداد واقعی Flux
reasoning/code	۶ + ۴ سؤال از آزمایش تاریخی؛ regression هستند
تست کد	نمونه‌های عمومی یکسان؛ ۱۰ تست اضافه فقط برای evaluator
حجم اجرای واقعی	۲۲ تماس ۳۲ + v5 تماس ۵۴ = v4 تماس
تکرار	یک پاسخ اولیه در هر arm؛ فقط repair مجاز اضافه شد

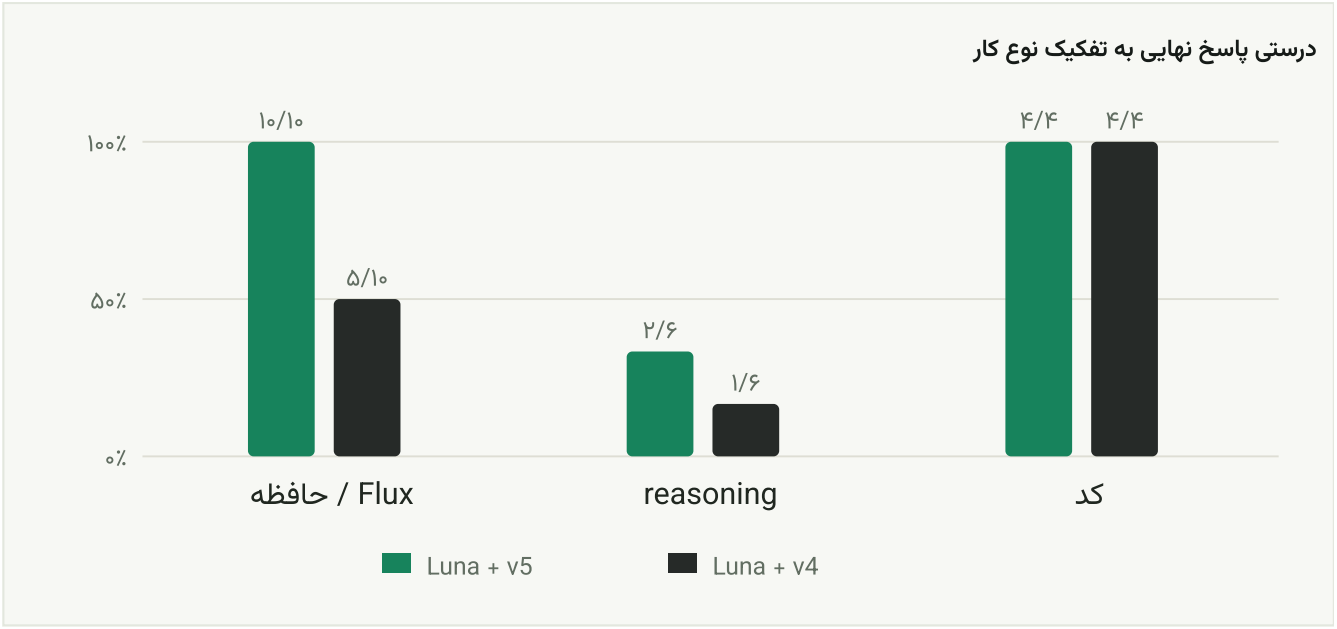
> همهٔ ۵۴ تماس واقعی تکمیل شدند و tool event نداشتند. Sol در این pilot استفاده نشد. این طراحی اثر prompt، بسته و verifier را با هم می‌سنجد؛ اثر جداگانهٔ هر جزء را تعیین نمی‌کند.

runtimeهای v3/v4/v5 و برنامهٔ pilot پس از انجماد hash یکسان داشتند. outputs، evaluation-only، public-jobs و provider_runs جدا نگه داشته شدند. [S8, S9]

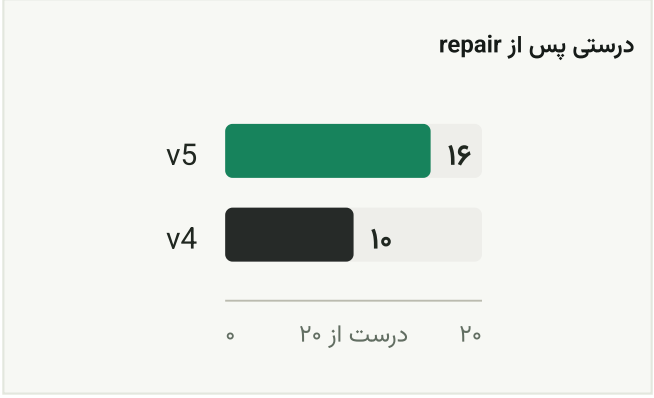
نتیجه‌ها: بهبود حافظه، محدودیت استدلال

در اجرای واقعیِ بیست‌سؤالی، v5 شانزده پاسخ درست و v4 ده پاسخ درست داشت. این اختلاف شش پاسخ، عمدتاً به بازیابی و قرارداد دقیق‌تر شواهد مربوط بود. نتیجهٔ مهم‌تر برای فرض اولیه آن است که بهبود حافظه، مشکل استدلال مستقل را حل نکرد: هر دو مسیر هنوز روی چند سؤال ریاضی اشتباه کردند. بنابراین داده‌ها از برتری این نسخه در این پایلوت پشتیبانی می‌کنند، اما هم‌سطح‌شدن توان عمومی Luna با Sol را ثابت نمی‌کنند.

v5 در حافظه بهتر بود؛ reasoning هنوز مسئله دارد







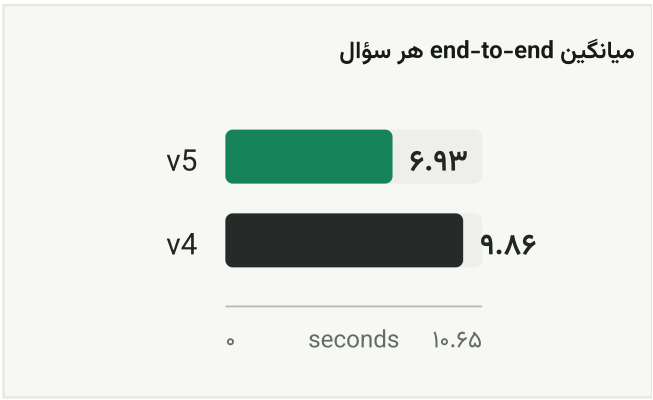
کیفیت مستقل از verifier	Luna + v5	Luna + v4
پاسخ درست نهایی	۱۶ / ۲۰ = ۸۰٪	۱۰ / ۲۰ = ۵۰٪
تولید اول	۱۴ / ۲۰ = ۷۰٪	۱۱ / ۲۰ = ۵۵٪
سؤال‌هایی که فقط این arm درست بود	۶	۰
native verifier در reasoning	۶ مورد unverified	می‌تواند correct را رد کند

۳۰ واحد درصد بهبود در این pilot مشاهده شد؛ «۳۰٪ افزایش هوش عمومی» نیست. تمام ۴ پاسخ کدِ هر arm تست‌ها را پاس کردند، ولی در reasoning فقط ۲/۶ و ۱/۶ درست بود.

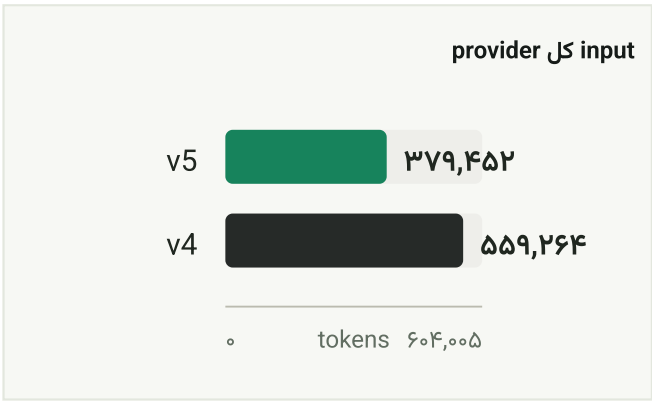
چهار شکست مشترک v5: احتمال شرطی، CRT، انتظار الگوی تاس و کوتاه‌ترین مسیر. این خطاها با RAG بهتر به‌خودی‌خود حل نشدند؛ دادهٔ مسئله از ابتدا در سؤال موجود بود.

نتایج score از raw output دوباره محاسبه شدند. «verified» بومیِ نسخه و «جواب درست» evaluator یکی نیستند. ۲۰ سؤال و یک نوبت، ادعای برتری عمومی یا هم‌سطح شدن با Sol را پشتیبانی نمی‌کند. [S8, S9]

repair کمتر، تماس کمتر، input کمتر









سنجهٔ همین اجرای واقعی	v5	v4
میانهٔ زمان سؤال	۶٫۲۵ s	۹٫۹۱ s
میانگین evidence نهایی	۱۹۶٫۳ tokens	۵۷۱٫۸ tokens
output کل provider	۱۴۶۲ tokens	۱۶۷۴ tokens
scaffold / cache سهم	در receipt ثبت شده؛ pack نیست	در receipt ثبت شده؛ pack نیست

> در این pilot: میانگین زمان ۲۹٫۶۹٪ کمتر، input کل ۳۲٫۱۵٪ کمتر و تعداد تماس ۳۱٫۲۵٪ کمتر بود. زمان صف و ساخت اولیهٔ index در end-to-end هر job نیست؛ اجرا دو worker هم‌زمان داشت.

بستهٔ v5 در workload offline بزرگ‌تر بود، اما در pilot مدل کوچک‌تر شد؛ مجموعهٔ سؤال، route و رشد بستهٔ repair v4 متفاوت بودند. این دو مقایسه را ادغام نمی‌کنیم. هزینهٔ دلاری نامعلوم است. [S7, S8]

پاسخ‌های واقعی؛ پشت هر عدد چه اتفاقی افتاد؟

میانگین‌ها بخشی از داستان را پنهان می‌کنند. برای نمونه، v4 در بعضی سؤال‌ها ابتدا مقدار درست را تولید کرد، اما verifier آن را رد کرد و repair پاسخ را به null تغییر داد. از سوی دیگر، چند پاسخ ریاضی در هر دو نسخه غلط ماندند. جدول‌های زیر خودِ خروجی نهایی را کنار مرجع قرار می‌دهند تا روشن باشد کدام بهبود از حافظه آمده، کدام از کنترل پاسخ و کجا محدودیت مدل همچنان باقی است.

حافظه و قرارداد واقعی Flux

در این ۱۰ سؤال، v5 تمام پاسخ‌ها را درست داد و v4 پنج پاسخ درست داشت. مقدارها عین خروجی نهایی‌اند؛ «null» پاسخ نامعلوم است، نه جواب صحیح برای منبعی که مقدار مشخص دارد.

سؤال	Luna + v5	Luna + v4	مرجع
حقیقت string `holdfinal-f0`	درست `"group_759"`	درست `"group_759"`	`"group_759"`
حقیقت integer `holdfinal-f1`	درست `350`	درست `350`	`350`
مجموعهٔ literal `holdfinal-f2`	درست `["enabled_2","disabled_2"]`	غلط `null`	`["enabled_2", "disabled_2"]`
عدد همراه واحد `holdfinal-f3`	درست `{"amount":253,"unit":"ms"}`	غلط `null`	`{"amount": 253, "unit": "ms"}`
alias انگلیسی `holdfinal-q0`	درست `"setting_value_796598"`	غلط `null`	`"setting_value_796598"`
alias فارسی `holdfinal-q1`	درست `"setting_value_583388"`	درست `"setting_value_583388"`	`"setting_value_583388"`
مالک dependency ۱ `holdfinal-h0`	درست `"team_0"`	غلط `null`	`"team_0"`
مالک dependency ۲ `holdfinal-h1`	درست `"team_1"`	غلط `null`	`"team_1"`
Flux: max_concurrency `Flux-limit`	درست `2`	درست `2`	`2`
Flux: allowed owner `Flux-owner`	درست `["local-user"]`	درست `["local-user"]`	`["local-user"]`

> درخواست‌های مربوط به allowed owner، مجموعهٔ مجاز `Literal` را می‌پرسیدند؛ این پاسخ اثبات مالک واقعی یا وضعیت runtime امروز نیست.

JSON کامل با status، binding، citation، رسید و متن provider در outputs/ و results.json ذخیره شده است. مقدارهای مصنوعی، حقایق اختصاصی benchmark هستند. [S8]

reasoning و کدنویسی با شواهد یکسان

در reasoning تمام دادهٔ لازم در سؤال بود. چهار پاسخ کد با نمونه‌های عمومی و caseهای اضافهٔ withheld اجرا شدند. تعداد تست پاس، به معنی اثبات کل رفتار ممکن برنامه نیست.

سؤال	Luna + v5	Luna + v4	مرجع
احتمال شرطی R00	غلط 0.16864	غلط 0.168816	0.168722
عضویت در یک مجموعه R01	درست 222	غلط 295	222
رشتهٔ باینری R02	درست 792	درست 792	792
CRT R03	غلط 931	غلط 559	79
انتظار الگوی ۶۶۵ R04	غلط 258	غلط 258	216
کوتاه‌ترین مسیر R06	غلط 11	غلط 11	13
canonical revision C00	درست 6/6 tests	درست 6/6 tests	تست اجرایی مستقل
weighted intervals C01	درست 6/6 tests	درست 6/6 tests	تست اجرایی مستقل
shortest path code C02	درست 5/5 tests	درست 5/5 tests	تست اجرایی مستقل
merge intervals C03	درست 5/5 tests	درست 5/5 tests	تست اجرایی مستقل

> برای CRT، پاسخ مرجع ۷۹ است؛ مدل‌ها ۹۳۱ و ۵۵۹ دادند. انتظار الگوی ۶۶۵ برابر ۲۱۶ است؛ هر دو ۲۵۸ دادند. این خطاها مرز توان reasoning را نشان می‌دهند.

چهار جواب کد در هر arm تمام ۲۲ case عمومی/withheld متناظر را پاس کردند. این ۲۲، شمار caseها برای چهار سؤال است؛ نه ۲۲ مسئلهٔ مستقل. [S8, S9]

این کار چه چیزی برای Flux به ارمغان می‌آورد؟

مزیت قابل دفاع این مسیر، حافظه‌ای است که برای پاسخ‌های وابسته به پروژه شواهد مشخص، بودجهٔ کنترل‌شده و مسیر بررسی‌پذیر فراهم می‌کند. در پایلوت فعلی، تعداد فراخوانی‌ها و زمان متوسط نیز کمتر شد. این مزیت زمانی به مزیت رقابتی محصول تبدیل می‌شود که روی پروژه‌های بیشتر، سؤال‌های تازه و دادهٔ در حال تغییر تکرار شود. نسخهٔ فعلی یک candidate آزمایشی است و هنوز به Flux اصلی متصل نشده است.

مزیت رقابتی بالقوه؛ هنوز یک candidate مستقل

پیاده‌شده و آزموده‌شده	پیاده‌نشده / اثبات‌نشده
schema / JSON / AST و source binding	معنای آزاد کامل یا parser INI/YAML
یک hop ثبت‌شده و reference مشترک	note معنایی بین تمام یال‌ها؛ زنجیرهٔ دلخواه چند field
حذف محدود optionalها و پذیرش exact	optimizer خودآموز یا وزن کالیبره‌شده
router سه‌حالته، verifier typed، ledger	توان reasoning هم‌سطح Sol
adapter حساب موجود و اجرای واقعی Luna	اجرای production یا integration حافظهٔ اصلی
۵۰ تست و hash freeze و آرشیو شکست	benchmark خارجی بزرگ و تکرارشونده
proposal دارای hash/reference	تغییر پارامتر مدل یا ارتقای خودکار حافظه

۱

مزیت امروز

شاهد متصل به ادعا؛ trace قابل ممیزی

۲

آزمون بعدی

دادهٔ مستقل، چند تکرار، paraphrase بدون alias

۳

reasoning

checker عمومی معتبر و بررسی مدل قوی‌تر

۴

ادغام بعدی

پذیرش candidate، سپس integration جدا

نقشهٔ فایل‌های اجرا

فایل	وظیفه
contracts.py / memory_v5.py	قرارداد و snapshot / extraction / graph
engine_v5.py	router، ماتریس، retrieval و بودجه
verifier_v5.py / code_worker.py	proof و تست اجرایی محدود
controller_v5.py / provider_v5.py	repair، ledger و receipt واقعی
cli.py / test_v5.py	demo و ۵۰ آزمون
benchmark_v5.py / run_pilot.py	offline منجمد / آزمایش واقعی جفت‌شده

آنچه می‌تواند Edge رقابتی Flux بسازد: حافظهٔ اختصاصی درست، هزینهٔ تماس کمتر و پاسخ قابل‌ردیابی. آنچه هنوز نداریم: اثبات برتری عمومی reasoning یا موفقیت محصول در استفادهٔ واقعی.

هیچ مدل تازه‌ای آموزش داده نشد. «خودتخریبی RAG» در این مسیر، حذف از نمای مشتق‌شده بود و به معنی پاک کردن اصل دانش نبود. [S6, S7, S8]

منابع، قابلیت بازتولید و حدود ادعا

برای این مقاله، آزمایش تازه‌ای روی مدل انجام نشده است؛ اعداد و خروجی‌ها از رسیدهای ذخیره‌شده و فایل‌های نتیجه استخراج شده‌اند. باید میان طرح پیشنهادی، کد پیاده‌شده، آزمون offline و پاسخ واقعی مدل فرق گذاشت. مرجع‌های زیر این مرز را قابل بررسی می‌کنند و hash کامل آن‌ها در فایل همراه سند ثبت شده است.

هر ادعا به یک اجرای مشخص وصل است

شناسه	منبع / اجرای مرتبط	SHA-256 کوتاه
S1	v1: graph / diffusion	`3b5fecfcfb3ff758`
S2	v2: ارزش و shared mini-block	`7c1b20336f6d8af4`
S3	v3: نتیجهٔ واقعی Luna/Sol	`3e3af369e4c91bea`
S4	v4: holdout اصلی ۵۶تایی	`72e24ade0d55dfc8`
S5	بازبینی مستقل v4	`7104c8b50b917d22`
S6	طراحی مصوب v5	`0fac99f3b8a3f2cb`
S7	v5: holdout نهایی offline	`188f4e9f4cf21560`
S8	اجرای واقعی Luna + v4/v5	`cc2646dfbf0c11ce`
S9	بازسنجی و receipt verification	`f3b60e96b999bebd`

S1 `experiments\diffusion_memory_20261001\REPORT.fa.md`

S2 `experiments\diffusion_memory_v2_20261001\REPORT.fa.md`

S3 `experiments\request_memory_v3_20261002\results.json`

S4 `experiments\request_memory_v4_20261002\holdout-results.json`

S5 `experiments\request_memory_v4_review_20261002\review-results.json`

S6 `experiments\request_memory_v5_design_20261002\ALGORITHM.fa.md`

S7 `experiments\request_memory_v5_20261002\runs\final\holdout-results.json`

S8 `experiments\luna_v4_v5_live_20261002\results.json`

S9 `experiments\luna_v4_v5_live_20261002\verification.json`

چهار سطح ادعا

طراحی
هدف و قرارداد پیشنهادی

offline
retrieval، budget و checker

مدل واقعی
جواب و receipt تکمیل

production
هنوز اجرا نشده

فایل `evidence-manifest.json` کنار HTML شامل hash کامل منابع است. سند از نتیجه‌های ذخیره‌شده ساخته شد؛ برای تهیهٔ این گزارش هیچ تماس تازهٔ مدل یا تغییر runtime انجام نشد.

محدودیت‌های اصلی: دادهٔ مصنوعی و alias معلوم، فقط دو سؤال واقعی در pilot آخر، مجموعهٔ کوچک، یک تکرار، تفاوت schema/prompt، منابع canonical مشترک، دو worker هم‌زمان و هزینهٔ دلاری نامعلوم.
